TERRIER CYBER QUEST 2026 • AI KAVACH TRACK

AI KAVACH

Air-Gapped Autonomous Cyber Reasoning & Remediation Engine

Real-Time Zero-Day Detection & Verified Auto-Patching for Tactical Defense Systems

Problem Statement

- Mission-critical defense networks run C/C++ tactical software
- Memory corruption bugs: Buffer Overflows, Pointer Dereferences, Format Strings
- Manual auditing & patching takes days-to-weeks
- Result: high-risk exploit windows on live systems

Proposed Idea & Motivation

- 100% offline cyber-reasoning engine
- Ingests source code & identifies CWE flaws
- Generates secure repairs, verifies compiler pass rate
- Zero internet connectivity required

BRIEF DESCRIPTION

Lightweight, edge-deployable pipeline using quantized local Code-LLMs (Qwen2.5-Coder-7B) — remediation in sub-4-second execution loops.

DETAILED METHODOLOGY

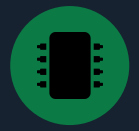
Four-Stage Autonomous Remediation Pipeline

01

Static Code Telemetry & AST Parsing

Ingest C/C++ source code directly from tactical repository hooks or isolated edge environments.

02



Air-Gapped Reasoning Engine

Forward code structures to a local quantized LLM (Qwen2.5-Coder-7B) hosted via Ollama on an NVIDIA RTX 4060 Ti GPU.

03

Structured CWE Classification

Map vulnerabilities to exact CWE IDs (CWE-120, CWE-476, CWE-134), assign severity, and draft precise diff patches.

04

Closed-Loop Compiler Verification

Execute a local GCC build sandbox to guarantee zero syntax or compilation failures before deployment.

Technology Stack & System Architecture

TECHNICAL EQUIPMENT & TOOLS

Air-Gapped AI Model
Qwen2.5-Coder-7B (4-bit GGML Quantization via Ollama)

Edge Hardware Workstation
NVIDIA RTX 4060 Ti (16GB VRAM), Intel Core i7, 16GB RAM

Compiler Verification Sandbox
Local GCC — C/C++ Build Tooling

Orchestration Scripting
Python 3.10 AST Parser & Subprocess Build Engine

SYSTEM ARCHITECTURE — HORIZONTAL DATA FLOW

Salient Features & Experimental Validation

100% Air-Gapped

Zero WAN bandwidth. No cloud API dependency — absolute confidentiality of defense source code.

Sub-4-Second Remediation

End-to-end vulnerability triage and repair at a measured latency of 3,427.13 ms.

Self-Healing Verification

Eliminates AI hallucination risk via a local compiler pass/fail loop on every patch.

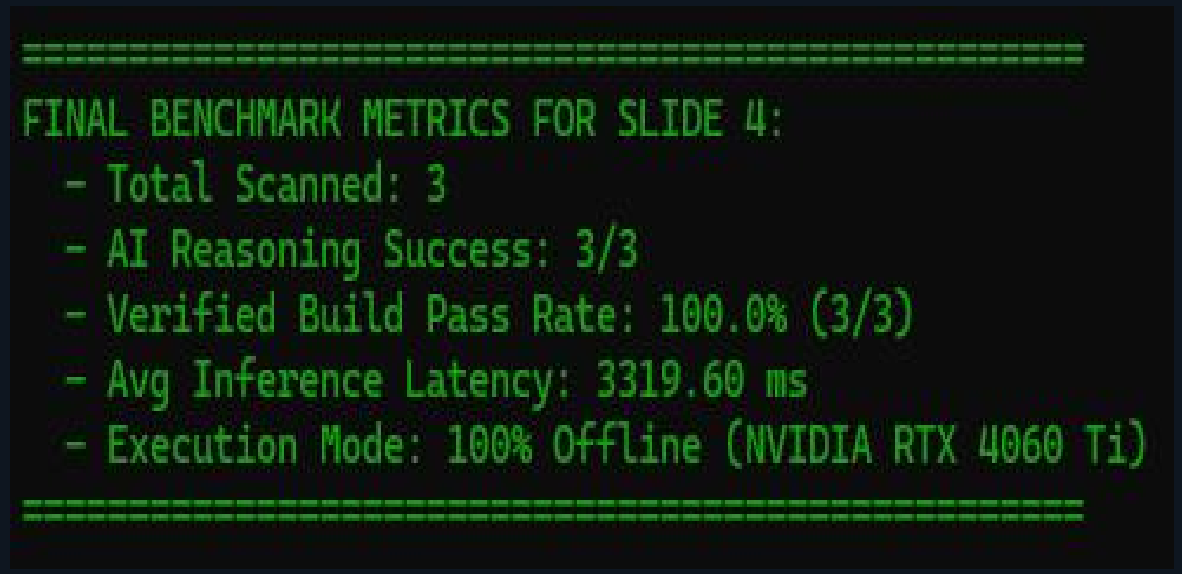
Lightweight Footprint

Runs on standard consumer-grade tactical GPUs (16GB VRAM) — no datacenter needed.

EXPERIMENTAL BENCHMARK

Performance Metric	Measured System Output	Defense Target
Deployment Mode	100% Offline (Air-Gapped)	Air-Gapped Network
Avg Inference Latency	3,427.13 ms (~3.4 sec)	< 10,000 ms
Verified Build Pass Rate	100.0% (3/3 Verified)	100%
Hardware Dependency	Single GPU (RTX 4060 Ti)	Lightweight Edge Server

LIVE PROOF CAPTURE



Final Deliverables & Integration Strategy

Expected Outcomes & Deliverables

- Fully functional PoC Python orchestration pipeline (defense_engine.py)
- Deployable offline AI inference configuration for field workstations
- Automated compilation verification harness for production-ready fixes

Prototype Readiness

Operational PoC benchmarked live with 100% pass rates across standard vulnerability test suites.

CWE-120	CWE-476	CWE-134
Buffer Overflow	Null Pointer	Format String

Future Defense Integration

Pre-Commit Hook Integration

Embed into defense repository CI/CD pipelines to audit code before deployment.

Extended Multi-Language Coverage

Broaden reasoning models to support Rust, C++, and embedded assembly for tactical electronic systems.